

Résumé du manuscrit de thèse

Protocoles pair à pair pour un apprentissage décentralisé efficace et résilient

Théorie, conception et évaluation

Mohamed Amine Legheraba

Sous la direction de Maria Potop-Butucaru et Sébastien Tixeul

Sorbonne Université — Laboratoire d'Informatique de Paris 6

Introduction

Les systèmes distribués à grande échelle sont aujourd'hui omniprésents, qu'il s'agisse de réseaux de diffusion de contenu, de systèmes de stockage décentralisés, ou plus récemment de plateformes d'apprentissage automatique collaboratif. Dans ce contexte, les architectures pair à pair occupent une place particulière : elles permettent de coordonner un grand nombre de nœuds sans recourir à une autorité centrale, ce qui les rend intrinsèquement résistantes aux pannes, scalables, et respectueuses de la confidentialité des données. Cependant, concevoir des protocoles pair à pair qui soient à la fois efficaces, robustes aux défaillances, et capables de supporter des charges applicatives complexes comme l'apprentissage automatique reste un défi ouvert.

Les réseaux pair à pair n'apparaissent pas dans le vide : ils sont le produit d'une longue histoire d'infrastructures de communication. L'Internet, né du désir de construire une infrastructure résiliente à la défaillance partielle, a été conçu dès l'origine pour qu'aucun nœud, institution ou gouvernement ne puisse contrôler le flux d'information. Pourtant, l'architecture ouverte et décentralisée de l'Internet n'a pas empêché l'émergence de la centralisation au niveau des services. Au fil de deux décennies, un petit nombre d'entreprises technologiques a occupé une position de domination structurelle sur la vie numérique de milliards de personnes. Leur succès reposait sur une logique simple : un service centralisé peut servir la planète entière tout en s'améliorant continuellement par l'agrégation des données de ses utilisateurs. Mais ces avantages ont eu un prix : le transfert progressif de données personnelles, de traces comportementales et, en fin de compte, d'une part d'autonomie individuelle vers des entités privées dont les décisions restent largement opaques.

En réponse à cette concentration, les protocoles pair à pair ont offert une vision alternative : des participants interagissent directement entre eux plutôt que par l'intermédiaire d'un coordinateur central. Des systèmes comme BitTorrent, Tor ou les blockchains ont démontré que les alternatives décentralisées étaient techniquement réalisables. Ils ont également soulevé des questions d'une profondeur et d'une généralité suffisantes pour attirer l'attention de la communauté de recherche : comment un système coordonne-t-il sans centre ? Comment reste-t-il fiable lorsque ses participants ne le sont pas ? Comment passe-t-il à l'échelle lorsqu'aucune entité ne supervise sa croissance ?

Sur une trajectoire parallèle, l'intelligence artificielle est devenue une ambition intellectuelle majeure depuis les premiers jours de l'informatique. Les modèles d'apprentissage automatique sous-tendent aujourd'hui une part croissante des systèmes qui structurent la vie quotidienne. Ce qui distingue les systèmes modernes d'apprentissage automatique des logiciels classiques, c'est leur nature

fondamentale : un modèle n'exécute pas des instructions explicites — il apprend à partir de données. L'entraînement d'un modèle d'apprentissage automatique est par nature une opération gourmande en ressources qui favorise la centralisation. Elle nécessite l'accès à de grands volumes de données, que les plateformes accumulent comme sous-produit de leur échelle, et à une infrastructure de calcul significative, que seules quelques organisations peuvent se permettre. La centralisation n'est donc pas une caractéristique accessoire du développement moderne de l'IA — c'est une tendance structurelle dans les conditions actuelles.

Pourtant, cette tendance n'est pas inéluctable. On peut légitimement se demander s'il est possible d'entraîner des modèles différemment — sans agréger les données brutes en un seul lieu, sans déléguer le contrôle à une autorité centrale, et sans exiger d'aucun participant qu'il expose des informations qu'il préférerait garder privées. C'est précisément la question centrale de cette thèse :

Est-il possible de construire un système d'apprentissage décentralisé qui égale l'efficacité du federated learning centralisé, tout en étant entièrement décentralisé et résilient aux pannes et aux participants malveillants ?

Les motivations pour poursuivre cette recherche vont bien au-delà de la curiosité technique. Les dimensions éthiques sont concrètes et pressantes. La confidentialité est en jeu, car l'entraînement centralisé exige que les données brutes quittent les mains de ceux qui les ont générées. La souveraineté est menacée par la dépendance croissante des institutions publiques à l'égard de modèles développés par un petit nombre d'acteurs privés. La responsabilité est compromise lorsqu'un modèle entraîné sur des données non divulguées, par un processus non reproductible et déployé via une interface opaque, offre peu de prise aux instruments classiques de contrôle démocratique. L'apprentissage décentralisé répond précisément à cette tension : la confidentialité et l'absence de contrôle central ne sont pas des propriétés qui dépendent de la bonne volonté d'un opérateur — elles sont des conséquences structurelles de la façon dont le système est conçu.

Cette thèse apporte quatre contributions originales à l'intersection des réseaux pair à pair et de l'apprentissage automatique décentralisé. La première est **Elevator**, un protocole d'échantillonnage pair à pair décentralisé qui organise les nœuds en un overlay hub-and-spoke par un mécanisme d'élection léger et auto-organisé. La deuxième est **Lift**, une extension d'Elevator qui traite la tolérance aux fautes byzantines par un mécanisme de redistribution déterministe des hubs. La troisième est **HEAL** (Hub Enhanced Adaptive Learning), un protocole d'apprentissage fédéré décentralisé construit directement sur l'overlay Elevator. La quatrième est **FLAIR** (Federated Learning with Adaptive Integrity-preserving Randomness), une adaptation de HEAL aux contraintes des réseaux sans fil physiques, qui valide la modularité de l'architecture en couches.

Modèle formel

Le manuscrit débute par l'introduction d'un modèle formel du système qui sert de socle à l'ensemble des contributions. Ce modèle adopte une approche ascendante, partant du nœud comme brique fondamentale du système, puis modélisant le réseau pair à pair formé par leurs interactions, et enfin analysant les phénomènes émergents qui se manifestent au niveau du réseau.

Nœud et réseau

Un nœud est défini comme une entité de calcul abstraite, modélisée comme une machine à états (S, s_0, δ) , où S est l'ensemble des états locaux possibles, $s_0 \in S$ est l'état initial, et $\delta : S \rightarrow S$ est la fonction de transition. Tous les nœuds sont supposés identiques en termes de capacités de mémoire et

de calcul, afin de se concentrer sur les interactions induites par le protocole plutôt que sur les disparités de ressources.

Le réseau d'overlay est modélisé comme un graphe $G = (V, E)$, où chaque sommet représente un nœud participant et les arêtes modélisent les relations de communication. Chaque nœud maintient une **vue partielle** $P(v) \subseteq V$, de taille bornée par une constante c avec $c \ll N$, qui correspond à son voisinage dans le graphe d'overlay. Un nœud ne peut envoyer un message qu'à un nœud dont il connaît l'adresse, c'est-à-dire à un voisin à distance 1 dans l'overlay.

Le réseau sous-jacent est supposé fiable : la transmission des messages est instantanée, et les messages ne sont ni perdus ni corrompus. Cette abstraction permet de se concentrer sur la conception et l'analyse de l'overlay.

Synchronisation et dynamique

L'exécution du système est structurée en **étapes de protocole** et **cycles de protocole**. Une étape de protocole est une exécution unique du pseudocode du protocole par un nœud. Un cycle de protocole est une ronde d'exécution logique dans laquelle chaque nœud exécute exactement une étape de protocole. Deux modèles d'exécution sont distingués : le modèle synchrone, où tous les nœuds exécutent leur étape simultanément, et le modèle séquentiel, où les nœuds s'exécutent les uns après les autres dans un ordre aléatoire.

La dynamique du réseau est capturée par la notion de **graphe temporel** $G = (V, E, T)$, où $V(t)$ est l'ensemble des sommets présents au temps t et $E(t)$ l'ensemble des arêtes présentes au temps t . Le churn est modélisé comme un phénomène temporellement localisé : pendant une période prédéfinie, une fraction donnée de nœuds est déconnectée et remplacée par de nouveaux nœuds, après quoi le réseau évolue sous le seul effet du protocole.

Modèles de pannes

Le modèle de pannes couvre trois scénarios distincts. Les **pannes franches** correspondent à l'arrêt définitif d'un nœud, modélisé par le modèle CSMP $\langle n, t \rangle [\emptyset]$. Les nœuds défaillants cessent toute activité et ne se rétablissent jamais. Le **churn** désigne la dynamique continue d'arrivées et de départs de nœuds. Les **fautes byzantines** représentent des participants malveillants capables d'envoyer des messages arbitraires ou incorrects, modélisées par le modèle BSMP $\langle n, t \rangle [\emptyset]$. Ce dernier modèle de faute est le plus général et le plus difficile à tolérer.

État de l'art : gestion des overlays pair à pair

Le chapitre suivant dresse un panorama de l'état de l'art sur les réseaux overlay pair à pair, en les classifiant en quatre familles complémentaires.

Réseaux structurés

Les réseaux structurés, typiquement implémentés par des tables de hachage distribuées (DHT), imposent une topologie bien définie permettant un routage déterministe en $O(\log N)$ sauts. Chord organise les nœuds en anneau avec des finger tables, Pastry utilise un routage par préfixes communs, et Kademlia emploie une distance XOR. Si ces protocoles garantissent un petit diamètre et une diffusion rapide, ils souffrent d'une fragilité face aux pannes et au churn, d'une vulnérabilité aux attaques byzantines, d'un surcoût de maintenance significatif, et d'un couplage étroit avec l'application sous-jacente. Aucun ne supporte nativement la notion de nœud hub, qui est le mécanisme central que nous cherchons à introduire.

Réseaux non structurés et peer sampling

Les protocoles de peer sampling, tels que Cyclon, Newscast, HyParView et leurs variantes, maintiennent des overlays aléatoires par des échanges périodiques de vues partielles entre paires de nœuds. Ils offrent une forte résilience aux pannes et au churn, un faible surcoût de maintenance, et une flexibilité applicative. Cependant, ils ne supportent pas l'agrégation globale — l'absence de structure rend l'agrégation structurée impossible — et ne permettent pas l'élection de hubs, car ils produisent délibérément des distributions de degré uniformes.

Réseaux à loi de puissance

Les protocoles construisant des topologies à loi de puissance, comme Phenix, SG-1, T-MAN ou VICINITY, introduisent délibérément une hétérogénéité de degré permettant l'émergence de nœuds hautement connectés. Ces topologies réduisent le diamètre et accélèrent la diffusion. Cependant, l'émergence de hubs dans le sens topologique ne se traduit pas en un mécanisme d'agrégation structurée : les nœuds de degré élevé ne sont pas conscients de leur statut, ne se portent pas volontaires pour des tâches d'agrégation, et ne sont pas explicitement utilisés comme agrégateurs par la couche applicative. Aucun protocole existant ne réalise simultanément la décentralisation complète, la stabilité sous churn, la résilience byzantine et l'élection explicite de hubs avec un rôle actif d'agrégation.

Protocoles résilients aux fautes byzantines

Les protocoles résilients aux attaques byzantines, tels que Brahms, Basalt, SecureCyclon ou AUPE, durcissent les protocoles de bavardage existants contre les participants adverses. Ils confirment que la résilience byzantine dans le peer sampling est réalisable, mais au prix d'un surcoût, d'une complexité accrue, et dans certains cas d'une remise en question partielle de la décentralisation. Plus important encore, tous héritent des limitations fondamentales du paradigme de bavardage : pas d'agrégation globale, pas de structure de hubs, et diffusion lente.

Synthèse

Le tableau comparatif établi dans le manuscrit montre qu'aucune famille existante ne satisfait simultanément l'ensemble des critères requis pour l'apprentissage décentralisé : petit diamètre, diffusion rapide, résilience aux pannes et au churn, résilience byzantine, faible surcoût, décentralisation complète, flexibilité applicative, et élection de hubs. Cette lacune définit l'espace de conception qu'Elevator et Lift sont conçus pour occuper.

Elevator et Lift

Elevator constitue la première contribution originale de la thèse. C'est un protocole d'overlay pair à pair qui organise le réseau en une structure à deux niveaux par l'émergence dynamique de nœuds hubs.

Service d'échantillonnage de hubs

Elevator introduit une nouvelle abstraction : le **service d'échantillonnage de hubs**, une généralisation du service de peer sampling traditionnel. Au lieu de retourner des paires aléatoires arbitraires, il vise à permettre l'émergence et le maintien aléatoire d'un nombre contrôlé de nœuds hubs dans l'overlay.

Un **hub** est formellement défini comme un nœud $h \in V$ dont l'identifiant apparaît dans la vue partielle de chaque nœud du réseau : $\forall v \in V, h \in P(v)$. Autrement dit, un hub est un nœud auquel tous les autres nœuds sont directement connectés.

Un **service d'échantillonnage de hubs décentralisé** est un protocole qui, à partir d'une configuration d'overlay arbitraire, induit l'émergence d'un sous-ensemble $H \subseteq V$ de taille h tel que chaque

nœud de H satisfait la propriété de hub. Idéalement, l'ensemble résultant est tiré selon une distribution uniforme sur tous les sous-ensembles de V de taille h .

Description du protocole Elevator

Elevator hybride deux concepts fondamentaux : l'attachement préférentiel et l'attachement aléatoire. L'attachement préférentiel dicte que les nouvelles connexions sont établies de préférence avec les nœuds possédant le plus grand nombre de connexions existantes. L'attachement aléatoire assure que les nœuds maintiennent des connexions avec un sous-ensemble représentatif et diversifié du réseau.

Le protocole utilise les paramètres suivants : c , la taille maximale de la vue partielle (valeur par défaut 20) ; h , le nombre de connexions préférentielles (valeur par défaut $\frac{c}{2}$) ; et `maxsize_buffer_backward`, le nombre maximal de connexions entrantes à transmettre (valeur par défaut 100). Chaque nœud maintient un cache (liste des connexions sortantes) et une liste de `backward_peers` (nœuds ayant tenté de se connecter au nœud).

À chaque cycle, le protocole exécute les actions suivantes :

1. **Collecte des voisins à distance deux** : chaque nœud collecte les listes de voisins de ses voisins.
2. **Construction de la carte de fréquence et sélection des nœuds préférés** : le nœud construit une liste ordonnée des pairs les plus fréquents et contacte les h premiers, appelés nœuds préférés.
3. **Réception des listes entrantes** : chaque nœud préféré contacté renvoie un sous-ensemble de sa liste de `backward_peers` et ajoute le nœud demandeur à sa propre liste.
4. **Réinitialisation du cache** : le nœud réinitialise son cache à un tableau vide.
5. **Remplissage du cache** : le nœud sélectionne les nœuds préférés et $c - h$ pairs aléatoires issus de la liste des `backward_peers` reçus.

Ce mécanisme crée une dynamique de renforcement : les nœuds les plus connectés attirent davantage de connexions, ce qui accroît encore leur degré entrant. En quelques cycles seulement, cette dynamique fait émerger spontanément h nœuds hubs auxquels la quasi-totalité des nœuds du réseau est directement connectée. Le statut de hub n'est pas attribué explicitement : il est la conséquence observable d'un processus collectif décentralisé.

Analyse théorique

L'analyse théorique d'Elevator établit plusieurs résultats formels sous des hypothèses simplificatrices (réseau sans pannes, exécution synchrone, identifiants aléatoires, topologie initiale en graphe k -out aléatoire).

La **stabilité** est démontrée : une fois le réseau convergé vers h hubs, l'ensemble des hubs reste inchangé avec une probabilité élevée. La preuve repose sur trois points : (i) le nombre de hubs ne peut excéder h car chaque nœud sélectionne au plus h nœuds préférés ; (ii) le nombre de hubs ne peut décroître car la probabilité qu'un nœud aléatoire apparaisse dans les vues de tous les voisins d'un nœud est extrêmement faible (approx $(\frac{c-h}{N})^c$) ; (iii) l'ensemble des hubs reste le même car la probabilité qu'un nœud aléatoire remplace un hub existant est négligeable pour $c \ll N$ et $h \ll N$.

La **convergence** est établie par une séquence de propositions : si le réseau contient au moins un hub, il est fortement connexe avec forte probabilité ; un hub supplémentaire finit par apparaître ; le nombre de hubs ne peut décroître entre 1 et $h - 1$; par conséquent, le réseau converge vers un état stable contenant exactement h hubs.

Le **temps de convergence** est modélisé par deux modèles. Le modèle A (suite géométrique) suppose une croissance exponentielle du degré entrant, donnant un temps de convergence en $O(\log(\frac{N}{i}))$.

Le modèle B (fonction logistique à taux dynamique) introduit un effet de saturation lorsque le degré entrant approche la taille du réseau, et s'ajuste mieux aux données de simulation. L'évaluation quantitative montre que le modèle logistique présente systématiquement un RMSE et un MAE inférieurs au modèle géométrique, et que les temps de convergence prédits sont proches des résultats de simulation (typiquement 5 à 6 cycles pour $N = 1000$).

Évaluation par simulation

L'évaluation par simulation est conduite sur PeerSim avec un réseau de 1 000 nœuds, en comparant Elevator à Newscast, PROOFS et Phenix, sur 1 000 cycles et 100 répétitions.

En conditions nominales, Elevator produit un réseau de diamètre 2, avec un coefficient de clustering d'environ 0,6 et une longueur moyenne de chemin inférieure à 2. Ces propriétés sont comparables à celles de Phenix et nettement meilleures que celles de PROOFS (diamètre 3) et Newscast (diamètre 4).

La résilience aux pannes franches est évaluée en déconnectant 50 % des nœuds au cycle 500. Elevator n'est pas affecté : les hubs restants maintiennent un diamètre de 2 et les métriques structurelles restent stables. En cas de crash ciblé des hubs, de nouveaux hubs émergent naturellement en quelques cycles, confirmant la propriété d'auto-guérison du protocole.

Sous churn (10 % de nœuds remplacés par cycle pendant 500 cycles), le diamètre peut temporairement atteindre 3,25 mais les métriques restent meilleures que celles des protocoles de bavardage. Après la fin du churn, le réseau retrouve rapidement ses propriétés nominales.

Résilience aux attaques byzantines et protocole Lift

L'évaluation de la résilience byzantine révèle qu'Elevator résiste aux attaques individuelles et non coordonnées : un seul nœud byzantin actif ne devient hub que dans 7 % des simulations. Cependant, les nœuds byzantins coordonnés — qui partagent des informations mutuelles dans leurs réponses — constituent une menace critique. À partir de 2 % de nœuds byzantins coordonnés, la proportion de hubs byzantins augmente de manière drastique, et à 5 %, les 10 hubs deviennent byzantins.

Le protocole **Lift** répond à cette vulnérabilité par un mécanisme de redistribution déterministe des hubs. Après la phase de convergence initiale d'Elevator, chaque nœud correct construit une graine en concaténant les identifiants des h hubs courants, initialise un générateur de nombres pseudo-aléatoires (PRNG) avec cette graine, et génère un nouvel ensemble de h identifiants de hubs. Comme tous les nœuds corrects disposent de la même graine et du même PRNG, ils dérivent indépendamment un ensemble identique de nouveaux hubs, retirant ainsi aux nœuds byzantins toute capacité d'influence sur l'élection.

L'évaluation de Lift montre qu'à 5 % de byzantins, la contre-mesure est très efficace : le nombre moyen de hubs byzantins tombe à 0,34. À 10 %, l'efficacité diminue mais reste significative. À 15 %, les nœuds byzantins parviennent progressivement à regagner des positions de hub, ce qui définit la limite du mécanisme actuel.

Déploiement sur réseau TCP/IP

Une implémentation complète d'Elevator en Go utilisant le framework libp2p valide la faisabilité du protocole sur un réseau réel. L'implémentation inclut trois modes d'exécution (synchrone, synchronisé extérieurement, asynchrone) et a été testée sur des réseaux de 20 à 100 nœuds. Les résultats confirment l'émergence rapide de hubs en quelques cycles, la tolérance à l'exécution asynchrone et aux pannes de nœuds, et l'auto-guérison après crash de hubs.

État de l'art : apprentissage automatique décentralisé

Le chapitre suivant introduit les fondements théoriques de l'apprentissage automatique et de l'apprentissage fédéré décentralisé, constituant le pont entre la partie réseau et la partie applicative de la thèse.

Cadre formel

Un **nœud d'apprentissage** est un nœud pair à pair enrichi de deux composantes : un jeu de données local $\mathcal{D}_i$ (privé, jamais transmis) et un modèle local f_{θ_i} paramétré par $\theta_i \in \mathbb{R}^p$. L'objectif global du système est de minimiser collectivement la perte agrégée : $L_{\text{global}}(\theta) = \frac{1}{N} \sum_{i=1}^N L_i(\theta)$. Aucun nœud n'a accès à la perte complète, donc la minimisation doit être réalisée collaborativement par l'échange de paramètres de modèle entre voisins.

La convergence est définie en termes de la perte globale : le système a convergé s'il existe un temps T tel que $L_{\text{global}} \leq \varepsilon$ pour tout $t \geq T$. En pratique, on utilise la **précision globale** (moyenne des précisions locales) comme métrique complémentaire, ainsi que le **temps de convergence** vers un seuil de précision donné.

Stratégies d'agrégation

L'apprentissage fédéré classique (FedAvg) repose sur une topologie en étoile : un serveur central collecte les paramètres locaux de tous les clients, calcule leur moyenne pondérée, et rediffuse le modèle global. Bien que cette architecture permette une coordination efficace, elle introduit un point unique de défaillance et de confiance.

L'apprentissage fédéré multi-serveurs (Gaia) remplace le serveur unique par plusieurs serveurs formant un graphe complet, mais chaque serveur reste un point critique pour ses clients. L'apprentissage fédéré hiérarchique organise les serveurs en arbre, mais hérite des limitations des topologies arborescentes. L'apprentissage fédéré basé sur blockchain tente d'éliminer le coordinateur de confiance, mais les coûts de consensus et de stockage sont mal alignés avec la nature itérative et approximative de l'apprentissage distribué.

L'apprentissage par bavardage (gossip learning) est un paradigme entièrement décentralisé où les nœuds échangent des modèles par des interactions pair à pair aléatoires. Il offre une forte résilience mais converge significativement plus lentement que l'apprentissage fédéré en raison de la bande passante limitée des interactions locales. L'apprentissage épidémique étend le bavardage en exigeant que chaque nœud échange son modèle avec tous ses voisins à chaque ronde, ce qui accélère la convergence au prix d'un surcoût de communication et d'une sensibilité aux retardataires.

Tension fondamentale

La revue de la littérature met en évidence une tension fondamentale : les approches centralisées et hiérarchiques bénéficient d'une coordination efficace et d'une convergence rapide, mais reposent sur un point central de contrôle qui introduit fragilité, limitations de passage à l'échelle et exigences de confiance. Les approches entièrement décentralisées éliminent cette dépendance centrale et offrent de fortes propriétés de résilience, mais convergent typiquement plus lentement en raison de la bande passante limitée des interactions locales. Aucun protocole existant ne combine simultanément convergence rapide, résilience aux pannes et décentralisation complète. C'est précisément cette lacune que HEAL vient combler.

HEAL et FLAIR

Le sixième chapitre présente HEAL et FLAIR, les contributions applicatives de la thèse.

Architecture en couches de HEAL

HEAL est structuré comme une pile de protocole à quatre couches, inspirée du modèle OSI, conçue pour assurer la modularité, la séparation des préoccupations et l'évolutivité.

La **couche réseau** est responsable de la communication pair à pair de bas niveau, fournissant les primitives de passage de messages. Elle est équivalente à une pile TCP/IP standard.

La **couche overlay** est fournie par le protocole Elevator, qui détermine la topologie du réseau et l'identité des hubs à chaque cycle. Elle expose une API comprenant `getHub()`, `getHubs()`, `getRandomPeers()` et `isHub()`.

La **couche d'agrégation** définit la stratégie par laquelle les modèles localement entraînés sont combinés en un modèle global et redistribués. HEAL adopte une agrégation inspirée du Federated Learning, mais en distribuant la responsabilité d'agrégation par h coordinateurs parallèles — les hubs.

La **couche applicative** interface HEAL avec la tâche d'apprentissage concrète, sans imposer de contraintes structurelles sur le modèle au-delà de la possibilité d'être entraîné par descente de gradient et d'un format de paramètres compatible.

Protocole d'agrégation de HEAL

Le processus d'agrégation se déroule en cinq phases successives à chaque cycle :

1. **Entraînement local** : chaque nœud non-hub entraîne son modèle courant sur son jeu de données local.
2. **Transfert de modèle** : chaque nœud non-hub sélectionne s hubs uniformément au hasard et leur transmet son modèle local.
3. **Agrégation par hub** : chaque hub attend un délai configurable, puis calcule un agrégat local des modèles reçus par Average SGD.
4. **Coordination inter-hubs** : les hubs échangent leurs agrégats locaux entre eux (ils forment un graphe complet par construction d'Elevator) et calculent le modèle global par Average SGD. Le résultat est identique pour tous les hubs.
5. **Redistribution** : chaque hub transmet le modèle global aux nœuds qui lui ont soumis leur modèle. Chaque nœud conserve la première copie reçue.

Cette structure en deux niveaux d'agrégation — intra-hub puis inter-hub — est la propriété fondamentale qui distingue HEAL des approches de gossip learning à plat, et qui lui permet d'atteindre une convergence plus rapide en réduisant le nombre de cycles nécessaires à la propagation d'une mise à jour à travers l'ensemble du réseau.

Le surcoût de communication par cycle est de $2(n - h) \cdot s + h(h - 1)$ messages, comparable au Federated Learning ($2(n - 1)$) et bien inférieur à l'Epidemic Learning ($n \cdot c$).

Évaluation expérimentale de HEAL

L'évaluation est conduite sur un réseau de 100 nœuds en combinant PeerSim (pour la couche overlay) et Gossipy (pour la couche d'apprentissage), avec des modèles réels entraînés via PyTorch.

Deux tâches d'apprentissage sont évaluées : une classification binaire sur Spambase (régression logistique, 57 paramètres) et une classification multinomiale sur MNIST (LeNet5, 60 000 paramètres). Les données sont partitionnées de manière IID à travers les nœuds. HEAL est comparé à Federated

Learning, Gaia, Gossip Learning, Epidemic Learning, Epidemic Learning sur Chord, Epidemic Learning sur anneau, et FedLay.

En conditions nominales, HEAL atteint une précision finale de 0,90 sur Spambase et 0,97 sur MNIST, comparable au Federated Learning (0,91 et 0,97) et significativement supérieur au Gossip Learning (0,83 et 0,71). En termes de vitesse de convergence sur MNIST, HEAL ($s = 1$) atteint 0,95 de précision en 76 cycles, plus rapidement que toutes les méthodes de référence de type bavardage. Avec 7 hubs et $s = 3$, ce temps chute à 33 cycles, rendant HEAL 2,3 fois plus rapide que le deuxième meilleur résultat. Le nombre de messages échangés par cycle (210) reste proche du Federated Learning (198) et bien en deçà de l'Epidemic Learning (1 000).

En conditions de crash, HEAL maintient une précision finale de 0,97 même après la défaillance de 20 % des nœuds, et reste fonctionnel jusqu'à un niveau de crash de 50 %. Les attaques ciblées contre les hubs — y compris la défaillance simultanée de tous les 5 hubs — n'ont aucun impact mesurable sur la précision, grâce au mécanisme de réélection des hubs fourni par l'overlay Elevator. Cela confirme que HEAL n'a pas de point unique de défaillance, contrairement au Federated Learning et à Gaia.

Sous churn, HEAL subit une baisse temporaire de précision pendant la phase de churn, mais récupère à son niveau antérieur presque immédiatement après la fin du churn. Même à un taux de churn de 30 %, la précision finale au cycle 200 reste supérieure à 0,95.

Protocole FLAIR

FLAIR valide la modularité de l'architecture en couches en l'instanciant dans un contexte opérationnel fondamentalement différent : les réseaux sans fil ad hoc à ressources contraintes.

FLAIR diffère de HEAL sur trois couches. La couche réseau fonctionne en WiFi (IEEE 802.11) plutôt que TCP/IP. La couche overlay utilise un protocole d'élection de têtes de grappe (cluster heads) inspiré de LEACH, où chaque nœud calcule un seuil basé sur ses ressources (CPU, RAM, GPU, bande passante) et un tirage aléatoire via une fonction aléatoire vérifiable (VRF). Les nœuds élus diffusent des annonces, et les nœuds non-élus rejoignent la grappe dont le coût de communication est minimal. Les têtes de grappe tournent à chaque ronde, assurant un équilibrage probabiliste de la charge. La couche d'agrégation effectue l'agrégation localement au sein de chaque grappe, sans coordination inter-grappes. La convergence globale émerge progressivement de la rotation des têtes de grappe : les modèles agrégés localement sont redistribués à travers le réseau au fil des rondes successives.

L'évaluation sur le simulateur ns-3 valide ces propriétés. En réseau statique, FLAIR dépasse toutes les méthodes de référence en précision finale. Sous défaillances extrêmes (jusqu'à 90 % de nœuds défaillants), il maintient une précision supérieure à 0,85. Sous cinq modèles de mobilité, la précision reste à moins de 2 % du résultat de référence statique. Dans un scénario d'agriculture intelligente (80 capteurs fixes et 20 robots mobiles), FLAIR approche le résultat de référence centralisé (71,9 %) avec des précisions finales de 71,2 % et 71,4 %.

Conclusions et perspectives

Synthèse des contributions

Les contributions de cette thèse établissent un cadre cohérent pour l'apprentissage automatique décentralisé, efficace et résilient sur réseaux pair à pair. La progression de l'état de l'art, à travers Elevator, jusqu'à HEAL, Lift et FLAIR, constitue une réponse rigoureuse à la question fondatrice : l'agrégation structurée peut être réalisée sans centralisation, à condition que la couche d'overlay soit conçue dans ce but.

Elevator ouvre la voie à une nouvelle classe d’algorithmes — les algorithmes d’échantillonnage de hubs — où la centralité structurelle est délibérément construite au sein d’overlays non structurés. HEAL démontre que cette structure peut être exploitée pour récupérer l’efficacité du Federated Learning tout en préservant la décentralisation. Lift renforce la sécurité de l’overlay contre les attaques byzantines coordonnées sans compromettre la décentralisation. FLAIR valide que l’architecture en couches est véritablement modulaire : en substituant indépendamment les couches réseau, overlay et agrégation, un protocole adapté à un contexte radicalement différent émerge naturellement des mêmes fondations architecturales.

Limitations

Plusieurs limitations doivent être reconnues. L’analyse théorique d’Elevator repose sur des hypothèses idéalisées (absence de pannes pendant la construction de l’overlay, réseau fiable). Les simulations d’Elevator n’atteignent que 1 000 nœuds, alors qu’un protocole pair à pair à usage général devrait idéalement être validé à des échelles de dizaines ou centaines de milliers de nœuds. Les expériences HEAL sont limitées à 100 nœuds et à des modèles relativement simples. Chaque configuration expérimentale n’a été évaluée que sur 5 exécutions, ce qui peut être insuffisant pour capturer pleinement la variance. Les benchmarks ML (LeNet5/MNIST, régression logistique/Spambase) sont en deçà des standards actuels. L’hypothèse IID est omniprésente et remet largement en cause la pertinence pratique. Aucune garantie formelle de convergence n’est établie pour HEAL : l’observation que le protocole atteint des niveaux de précision compétitifs est empirique. La synchronisation implicite des hubs dans la phase inter-hub réintroduit une forme de coordination globale en tension avec l’objectif de décentralisation totale. Le protocole Lift ne traite que la couche overlay : la couche apprentissage reste vulnérable aux attaques par empoisonnement et aux attaques de confidentialité.

Perspectives

Les perspectives de recherche s’étendent sur plusieurs horizons temporels. À court terme, l’extension du mécanisme d’élection d’Elevator pour incorporer les capacités des nœuds (CPU, bande passante) est déjà en cours d’investigation, avec des résultats préliminaires encourageants. L’adaptation de la stratégie d’agrégation de HEAL pour atténuer l’hétérogénéité des données (scénario non-IID) est une extension naturelle et pressante. Le développement d’un nouveau simulateur unifié en Python, intégrant peer sampling, évolution dynamique du réseau et charges d’apprentissage automatique, est également en cours.

À moyen terme, l’intégration de la robustesse contre les attaques byzantines dans HEAL lui-même — par des règles d’agrégation robustes, la confidentialité différentielle, ou des protocoles d’agrégation sécurisée — est une étape nécessaire vers le déploiement en environnements hostiles. Un déploiement à grande échelle d’Elevator et HEAL sur un réseau réel permettrait de valider les hypothèses de passage à l’échelle. L’évaluation de HEAL avec des modèles à grande échelle (centaines de millions de paramètres) exposerait les limitations fondamentales en termes de bande passante, de latence d’agrégation et de besoins mémoire.

À plus long terme, l’extension du cadre à l’apprentissage non supervisé et par renforcement, l’adaptation au federated learning vertical, l’établissement de garanties formelles de convergence pour HEAL sous conditions réalistes, et la conception de mécanismes d’incitation pour récompenser la participation active et dissuader le parasitisme constituent des défis fondamentaux. Un objectif appliqué à long terme est le déploiement de la pile complète dans un cas d’usage industriel concret, tel

que les véhicules autonomes ou les essaims de drones, où les exigences de mobilité, de latence et de qualité du modèle sont particulièrement contraignantes.

Plus largement, cette thèse devrait être lue non comme une contribution incrémentale à un corpus existant, mais comme l'ouverture d'un nouveau territoire de recherche. En combinant une couche de peer sampling inédite avec une architecture d'apprentissage modulaire et agnostique à l'application, le cadre s'écarte à la fois de la tradition du federated learning — qui a largement tenu la centralisation pour acquise — et de la tradition du gossip learning — qui a accepté l'efficacité d'agrégation limitée comme une contrainte inhérente. L'espace de conception qui en résulte, dans lequel la structure de l'overlay et la performance de l'apprentissage sont co-conçues dès la base, n'avait pas, au meilleur de la connaissance de l'auteur, été exploré de manière rigoureuse avant ces travaux.